View Coordinate Architecture Migration Plan

Tracking document for migrating from the legacy `KometNodeFactory` / `ViewProperties` pattern to the `KlView.Factory` / `KlContext` architecture in the knowledge-layout subproject. This refactoring spans `framework`, `kview`, and `knowledge-layout`.

Table of Contents

Background: Two Architectures, One Transition . . . 1
- The intended design . . . 1
- The knowledge-layout system already solves this . . . 2
- What `KometNodeFactory` gets wrong . . . 2
- The collision point (ClassCastException) . . . 3

Phase 0: Immediate Fixes (unblock current build) . . . 3
Phase 1: Deprecate `KometNodeFactory` . . . 3
- KometNodeFactory implementations inventory . . . 4

Phase 2: Migrate `EntityKlWindowFactory` off `ViewProperties` . . . 4
- EntityKlWindowFactory implementations . . . 5

Phase 3: Migrate `JournalController` view coordinate wiring . . . 5
Phase 4: Migrate remaining `KometNodeFactory` consumers . . . 5
Phase 5: Simplify `ViewProperties` and observable view classes . . . 6
Phase 6: Remove deprecated code . . . 6
Reference: Key file locations . . . 6
- Framework (old architecture) . . . 6
- Knowledge-layout (new architecture) . . . 7
- Kview (boundary / migration target) . . . 7

Background: Two Architectures, One Transition

The intended design

The application needs a hierarchy of view coordinates:

```
Knowledge Base  (root defaults)
```

```
    └── Session    (user preferences)
        └── Journal  (journal-level overrides, e.g. time-travel)
            └── Window   (window-level overrides)
                └── Pane  (pane-level overrides, e.g. historical vs current)
```

Each level can override STAMP coordinates, language, logic, navigation, or edit coordinates. Fields not overridden are inherited from the parent. This enables capabilities like time-travel (one window historical, another current), resolving contradictions between authors or branches, and per-pane coordinate customization.

The knowledge-layout system already solves this

The `KlContext` + `KlView` architecture handles the multi-level override chain **without `ViewProperties` at all**. The mechanism is fundamentally different and cleaner:

`KlView` resolves coordinates by walking the JavaFX node tree:

```
KlView.context(node)
  → check node.properties[KL_CONTEXT]
  → walk up parent chain
  → check scene
  → check window
  → fallback: KnowledgeBaseContext
```

Each level (Window → Scene → Area → child Area) can have its own `KlContext` with its own `ObservableView viewCoordinate()`. A component calls `viewForContext()` or `calculatorForContext()` and gets the nearest context's coordinate — naturally inheriting from ancestors if no local context is set.

This is **exactly** the intended hierarchy (knowledge base → session → journal → window → pane) but implemented through the JavaFX scene graph rather than through a hand-wired `ViewProperties` parent chain. No casts. No `ObservableViewNoOverride` vs `ObservableViewWithOverride` distinction needed at the API boundary.

What `KometNodeFactory` gets wrong

Concern	KometNodeFactory (old)	KlView.Factory (new)
View coordinate source	Passed as `ObservableViewNoOverride` parameter — assumes flat 2-level hierarchy	Resolved from JavaFX tree via `KlContext` — naturally N-level
Factory parameter	Requires the root view + creates one override	No view parameter — component discovers its context after insertion into scene graph
Restoration	`@Reconstructor` + `Encodable.decode` reflection	`KlRestorable` + preference-based `FACTORY_CLASS_NAME`

Concern	KometNodeFactory (old)	KlView.Factory (new)
Lifecycle	None — constructor does everything	`knowledgeLayoutBind()` / `knowledgeLayoutUnbind()` separate setup from construction
Discovery	`ServiceLoader<KometNodeFactory>`	`PluggableService.forName()` from preferences

The key insight: `KometNodeFactory` **forces the view coordinate to be known at construction time and passed as a parameter**, which requires the caller to manually assemble the override chain. `KlView` defers coordinate resolution to the scene graph, which means the chain assembles itself.

The collision point (ClassCastException)

`ConceptKlWindow` sits at the boundary between architectures. It is called from the new pattern (`KlWindowPreferencesUtils.getJournalViewProperties()` builds a multi-level chain producing `ViewProperties` whose parent is an `ObservableViewWithOverride`) but internally delegates to the old pattern (`KometNodeFactory.create()` which demands `ObservableViewNoOverride`). The cast fails at runtime.

This is a symptom of the broader problem: the `ViewProperties` class conflates the **root view** (needed by old code) with the **immediate parent in the override chain** (needed by new code). With multi-level overrides, these are no longer the same object.

Phase 0: Immediate Fixes (unblock current build)

These are tactical fixes to get the application running while the broader migration proceeds.

- ☐ Fix `ConceptKlWindow` to not call `KometNodeFactory.create()` — construct `ConceptNode` directly with the `ViewProperties` it already has
- ☐ Audit all other `ClassCastException` risks where `parentView()` is cast to `ObservableViewNoOverride` (known location: `ExplorationNodeAbstract:300`)
- ☐ Verify the `makeOverridableViewProperties()` restoration on `ObservableViewBase` works correctly for all existing call sites

Phase 1: Deprecate `KometNodeFactory`

Mark the old factory interface and its ecosystem as deprecated with clear pointers to the replacement.

- ☐ Add `@Deprecated(forRemoval = true)` to `KometNodeFactory` interface (`framework/src/main/java/dev/ikm/komet/framework/KometNodeFactory.java`)
- ☐ Add deprecation Javadoc pointing to `KlView.Factory` / `KlArea.Factory`

- ☐ Add `@Deprecated` to `ExplorationNodeAbstract` (`framework/src/main/java/dev/ikm/komet/framework/ExplorationNodeAbstract.java`)
- ☐ Identify which of the 16 `KometNodeFactory` implementations are still actively used vs. legacy dead code

KometNodeFactory implementations inventory

Factory class	Module	Status
`SearchNodeFactory`	search	[ ] assess
`CompletionNodeFactory`	progress	[ ] assess
`ProgressNodeFactory`	progress	[ ] assess
`ListNodeFactory`	list	[ ] assess
`SetNodeFactory`	list	[ ] assess
`TableNodeFactory`	list	[ ] assess
`ConceptNodeFactory` (kview)	kview	[ ] assess
`ConceptDetaisNodeFactory`	details	[ ] assess
`DetailsNodeFactory`	details	[ ] assess
`ReasonerResultsNodeFactory`	classification	[ ] assess
`ConceptBuilderNodeFactory`	builder	[ ] assess
`PatternBuilderNodeFactory`	builder	[ ] assess
`ArtifactExportNodeFactory`	artifact	[ ] assess
`ArtifactImportNodeFactory`	artifact	[ ] assess
`GraphNavigatorNodeFactory`	navigator	[ ] assess
`PatternNavigatorFactory`	navigator	[ ] assess

Phase 2: Migrate `EntityKlWindowFactory` off `ViewProperties`

The `EntityKlWindowFactory` system in kview is the primary consumer of the problematic `ViewProperties` pattern. These factories should use `KlContext` instead.

- ☐ Refactor `EntityKlWindowFactory.create()` signature to accept a `KlContext` or `KlContextProvider` instead of `ViewProperties`
- ☐ Refactor `EntityKlWindowFactory.restore()` to resolve context from the scene graph rather than manually constructing `ViewProperties`
- ☐ Update `KlWindowPreferencesUtils.getJournalViewProperties()` — this is the method that builds the multi-level override chain manually; it should instead set `KlContext` on the appropriate JavaFX nodes
- ☐ Migrate `AbstractEntityChapterKlWindow` to use `viewForContext()` instead of storing `ViewProperties`

- ☐ Migrate `AbstractChapterKlWindow` similarly

EntityKlWindowFactory implementations

Factory	Status
`ConceptKlWindowFactory`	[ ] migrate
`PatternKlWindowFactory`	[ ] migrate
`GenEditingKlWindowFactory`	[ ] migrate
`LidrKlWindowFactory`	[ ] migrate
`GenPurposeKLWindowFactory`	[ ] migrate

Phase 3: Migrate `JournalController` view coordinate wiring

`JournalController` is the largest consumer of `makeOverridableViewProperties()`. It manually builds the override chain when creating windows, search panels, and navigation panels.

- ☐ Replace `journalViewProperties` field with a `KlContext` set on the journal pane
- ☐ Replace all `journalViewProperties.nodeView().makeOverridableViewProperties(…)` calls with `KlContext` placement on child nodes
- ☐ Migrate `loadSearchPanel()` — currently uses `KometNodeFactory` via `SearchNodeFactory`
- ☐ Migrate `loadClassicConceptNavigatorPanel()` — uses `KometNodeFactory` via `GraphNavigatorNodeFactory`
- ☐ Migrate `loadNavigationPanel()`
- ☐ Migrate all `createConceptWindow` / `createPatternWindow` / `createGenEditWindow` / `createLidrWindow` / `createGenPurposeKLWindow` methods
- ☐ Migrate `FilterOptionsPopup` creation (currently takes `ObservableViewBase`)
- ☐ Migrate `LandingPageController` similarly (uses same patterns)

Phase 4: Migrate remaining `KometNodeFactory` consumers

These are the other call sites and patterns that depend on the old factory.

- ☐ Migrate `TabGroup` (`framework/src/main/java/dev/ikm/komet/framework/tabs/TabGroup.java`) — uses `KometNodeFactory.getKometNodeFactories()` ServiceLoader for dynamic menu
- ☐ Migrate `@Reconstructor` pattern — preference-based restoration should use `KlRestorable` / `KlView.restore()` instead of `Encodable.decode()`
- ☐ Migrate activity stream setup — currently done by `KometNodeFactory.create()` default methods;

needs equivalent in `KlView` lifecycle

Phase 5: Simplify `ViewProperties` and observable view classes

Once the consumers are migrated, the framework view classes can be simplified.

- ☐ Evaluate whether `ViewProperties` is still needed or can be replaced entirely by `KlContext.viewCoordinate()`
- ☐ Evaluate whether `ObservableViewNoOverride` vs `ObservableViewWithOverride` distinction is still needed, or if a single `ObservableView` with optional parent binding suffices
- ☐ Remove the `parentView` / `rootView` complexity from `ViewProperties` if no longer needed
- ☐ Clean up deprecated `KlFactory` interface (`knowledge-layout/src/main/java/dev/ikm/komet/layout/KlFactory.java` — already marked `@Deprecated`)
- ☐ Clean up deprecated area types: `KlFieldArea`, `KlListOfFieldArea`, `KlAreaForEntity`

Phase 6: Remove deprecated code

After all consumers are migrated and a release has shipped with deprecation warnings:

- ☐ Remove `KometNodeFactory` interface and all 16 implementations
- ☐ Remove `ExplorationNodeAbstract`
- ☐ Remove `ViewProperties` class (if fully replaced)
- ☐ Remove `makeOverridableViewProperties()` from `ObservableViewBase`
- ☐ Remove `KlFactory` interface
- ☐ Remove `KlViewFactory` interface
- ☐ Clean up `module-info.java` service declarations

Reference: Key file locations

Framework (old architecture)

- `framework/src/main/java/dev/ikm/komet/framework/KometNodeFactory.java`
- `framework/src/main/java/dev/ikm/komet/framework/ExplorationNodeAbstract.java`
- `framework/src/main/java/dev/ikm/komet/framework/view/ViewProperties.java`
- `framework/src/main/java/dev/ikm/komet/framework/view/ObservableViewBase.java`
- `framework/src/main/java/dev/ikm/komet/framework/view/ObservableViewNoOverride.java`
- `framework/src/main/java/dev/ikm/komet/framework/view/ObservableViewWithOverride.java`
- `framework/src/main/java/dev/ikm/komet/framework/tabs/TabGroup.java`

Knowledge-layout (new architecture)

- `knowledge-layout/src/main/java/dev/ikm/komet/layout/KlView.java`
- `knowledge-layout/src/main/java/dev/ikm/komet/layout/KlArea.java`
- `knowledge-layout/src/main/java/dev/ikm/komet/layout/KlFactory.java` (deprecated)
- `knowledge-layout/src/main/java/dev/ikm/komet/layout/KlViewFactory.java`
- `knowledge-layout/src/main/java/dev/ikm/komet/layout/context/KlContext.java`
- `knowledge-layout/src/main/java/dev/ikm/komet/layout/context/KlContextFactory.java`
- `knowledge-layout/src/main/java/dev/ikm/komet/layout/context/KlContextProvider.java`
- `knowledge-layout/src/main/java/dev/ikm/komet/layout/window/KlJournalWindow.java`

Kview (boundary / migration target)

- `kview/src/main/java/dev/ikm/komet/kview/klwindows/EntityKlWindowFactory.java`
- `kview/src/main/java/dev/ikm/komet/kview/klwindows/AbstractEntityChapterKlWindow.java`
- `kview/src/main/java/dev/ikm/komet/kview/klwindows/AbstractChapterKlWindow.java`
- `kview/src/main/java/dev/ikm/komet/kview/klwindows/KlWindowPreferencesUtils.java`
- `kview/src/main/java/dev/ikm/komet/kview/klwindows/concept/ConceptKlWindow.java`
- `kview/src/main/java/dev/ikm/komet/kview/klwindows/concept/ConceptKlWindowFactory.java`
- `kview/src/main/java/dev/ikm/komet/kview/klwindows/pattern/PatternKlWindowFactory.java`
- `kview/src/main/java/dev/ikm/komet/kview/klwindows/genediting/GenEditingKlWindowFactory.java`
- `kview/src/main/java/dev/ikm/komet/kview/klwindows/lidr/LidrKlWindowFactory.java`
- `kview/src/main/java/dev/ikm/komet/kview/klwindows/genpurpose/GenPurposeKLWindowFactory.java`
- `kview/src/main/java/dev/ikm/komet/kview/mvvm/view/journal/JournalController.java`
- `kview/src/main/java/dev/ikm/komet/kview/mvvm/view/landingpage/LandingPageController.java`
- `kview/src/main/java/dev/ikm/komet/kview/controls/FilterOptions.java`
- `kview/src/main/java/dev/ikm/komet/kview/controls/FilterOptionsPopup.java`